Instructions

Homework 4

Focus

- Classes
- Pointers
- Association
- Oh, **no** heap.

Starter code:

Please note that we have provided a file, **hw04_start.cpp**. Of course, the file you will turn in will be **hw04.cpp**.

Problem:

We will [continue to] model a game of medieval times. Our world is filled with not only warriors but also nobles. Nobles don't have much to do except do battle with each other. (We'll leave the feasting and other entertainments for add-ons.) Warriors don't have much to do except hire out to a noble and fight in his behalf. Of course the nobles are pretty wimpy themselves and will lose if they don't have warriors to defend them. How does all this work?

- Warriors start out with a specified strength.
- A battle between nobles is won by the noble who commands the stronger army.
- The army's strength is simply the combined strengths of all its warriors.
- A battle is to the death. The losing noble dies as does his warriors.
- The winner does not usually walk away unscarred. All his men lose a portion of their strength equal to the ratio of the enemy army's combined strength to their army's. If the losing army had a combined strength that was 1/4 the size of the winning army's, then each soldier in the winning army will have their own strength reduced by 1/4. **(Yes, strength will need to be a double.)**

Hiring and Firing

- Warriors are hired and fired by Nobles. Lack of adequate labor laws, have left

the Warriors without the ability to quit, nor even to have a say on whether or not a Noble can hire them.
- However it is possible that an attempt to hire or fire may fail. Naturally the methods should not "fail silently". Instead, they will return true or false depending on whether they succeed or not.
- A Warrior can only be employed by one Noble at a time and *cannot* be hired away if he is already employed.
- As noted below, Nobles who are dead can neither hire nor fire anyone. (Note this will *implicitly* prevent dead Warriors from being hired.)
- When a warrior is fired, he is no longer part of the army of the Noble that hired him. He is then free to be hired by another Noble.
    - How do you remove something from a vector.
    - While there are techniques that make use of iterators, we have not yet discussed iterators so you will not use them here. (As a heads up, if you see a technique that requires you to call a vector's begin() method, that is using iterators. Don't use it.)
    - While it may seem a slight burden, certainly it does not require more than a simple loop to remove an item from a vector. No do not do something silly like create a whole new vector.
    - Soon we will cover iterators and then you will be freed from these constraints. Patience, please.

Death

It's a sad topic, but one we do have to address.

- People die when they lose a battle, whether they are a Nobles or Warriors.
- Nobles who are dead are in no position to hire or fire anyone. Any attempt by a dead Lord to hire someone will simply fail and the Warrior will remain unhired.
- However curiously, as has been seen before, Nobles can declare battle even though they are dead.
- Note that when a Noble is created he does not have any strength. At the same time he is obviously alive. So lack of strength and being dead are clearly *not* equivalent.

Output

A test program and output are attached. Note that the output shown is what you are expected to generate. Pardon us, we don't like limiting your creativity, but having your output consistent with ours makes the first step of grading a bit easier. And also helps you to be more confident that your code works.

Both the Noble and Warrior classes should have overloaded output operators. The implementation of the Noble's output operator should use / delegate to the Warrior's output operator.

Programming Constraints

What would a homework assignment be without unnecessary and unreasonable constraints?

Your classes must satisfy the following:

- The battle ***method*** will announce who is battling whom, and the result (as shown in the example output).
    - If one or both of the nobles is already dead, just report that.
      The "winner" doesn't win anything for kicking around a dead guy. And his warriors don't use up any strength.
    - Look at the output for the sample test program to see what you should be displaying.
- A noble's army is a **vector of pointers** to warriors. Warriors will be ordered in the army by the order in which they were *hired*. Be sure to maintain that order when a Warrior gets fired.

And some things to make life easier

This program does not read a file! All of the entities (i.e. Nobles and Warriors) are defined in the test code. Just thought we'd give you a break.

And also, just in case you are confused, let me point out that this problem does **not** involve the use of the heap. That means your program will **not** make use of the operator `new` or the operator `delete`.

We have not covered how to allow the Warrior class to also have a pointer back to the Noble who is employing him / her. **So do not do this!** I promise that we will cover how to handle that, soon, just not in time for this assignment.

Turn in

Hand in the file named **hw04.cpp**.

Submit **Cancel**